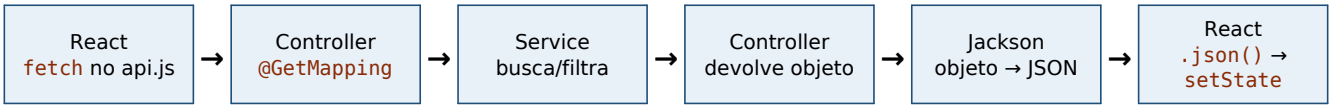
Esquema da prova — Spring Boot + React

DIAW · roteiro passo a passo · back-end na porta 8080, front-end na 5173

COMO OS DOIS LADOS CONVERSAM



A única ligação é o HTTP: o React chama uma URL, o Spring responde JSON. Os nomes dos campos do Model viram as chaves do JSON (por isso `monstro.nome` no React). Para ver a conversa: F12 → aba Network.

ANTES DE COMEÇAR

1	**Ler enunciado e README inteiros** Anotar quanto vale cada questão para dividir o tempo.
2	**Ler os arquivos fornecidos (Model e Service)** Anotar nome exato dos métodos públicos, ORDEM dos parâmetros e o que retornam (lista, `Optional`...).

BACK-END — SPRING BOOT

3	**Initializr** Maven · Java 25 · dependência **Spring Web** (só ela; Thymeleaf só se pedir HTML). Nada de JPA/banco sem a prova pedir: o projeto não sobe.
4	**Pastas e package** Application na RAIZ do pacote. Copiar Model/Service e dados para `resources`. Todo arquivo começa com `package` = caminho da pasta depois de `java/` com pontos. Pastas em minúsculo.
5	**Rodar vazio** Subir o projeto antes de escrever qualquer coisa. Se não sobe agora, não vai subir depois.
6	**Anotações do Controller** `@RestController` (JSON, não view) · `@RequestMapping("/api")` · `@CrossOrigin` para a porta do React · Service injetado pelo construtor.
7	**Endpoints** Dado no caminho → `@PathVariable`. Filtro opcional → `@RequestParam(required = false)`. Pode não existir → `ResponseEntity` com 404 (Optional: map ok / orElse notFound).
8	**Testar no navegador** Lista, lista filtrada, um id que existe e um que não existe (404). Só então ir para o React.

FRONT-END — REACT + VITE

9	**Criar e limpar** `npm create vite` (template react) → `npm install` → `npm run dev`. Apagar o exemplo do App.
10	**api.js** URL base em uma constante. Uma função por endpoint. **fetch não dá erro em 404**: conferir `resposta.ok` e lançar erro.
11	**Estados no App** Dados (listas) · carregando (começa true) · erro · o que o usuário controla: busca, filtro, item selecionado, aba.
12	**Carga inicial** `useEffect` com `[]` → busca → guarda no estado → `.catch` seta erro → `.finally` desliga o carregando.
13	**Lista filtrada = estado derivado** Calcular com `.filter()` no corpo do componente. **Nunca** em um useState separado. Busca e filtro juntos (&&).
14	**Componentes** Header → SearchBar → Filtro → Card → List. Estado no App, desce por props; eventos sobem por funções. Inputs controlados (value + onChange). Todo map com `key`.
15	**Tela** Carregando · erro · contagem "X encontrado(s)" · aviso de "nenhum encontrado".
16	**Modal de detalhes** Só existe se há item selecionado. `useEffect` com `[id]`. Fecha no × e no fundo; `stopPropagation` na caixa.
17	**Entidade relacionada (abas)** Aba = useState, sem router. IDs → nomes com mapa id → objeto. Ordenar uma CÓPIA: `[...lista].sort()`.

CSS

18	**Grade** `display: grid` + `repeat(auto-fill, minmax(220px, 1fr))` + gap. Responsivo sem media query.
19	**Cores por categoria** Classe gerada pelo nome sem acento: `normalize("NFD")` + remover acentos + minúsculo.
20	**Detalhes visuais** Hover nos cards · destaque por raridade/categoria · barras: trilho com `overflow: hidden` + barra com width em %.
21	**Modal** `position: fixed; inset: 0` + fundo escuro + flex centralizado na caixa.

DE ONDE VEIO ESSA PROP?

Componente usa algo que não criou → olhar as props na assinatura `function X({ ... })` → ir no arquivo que USA `<X prop={...} />` → repetir até chegar no `useState` ou no `fetch`.
Exemplo: `fetch` (api.js) → `setMonstros` (App) → `filtrados` → `<MonstroList monstros>` → `map` → `<MonstroCard monstro>`.

SE TRAVAR

Sintoma	Causa / correção
"blocked by CORS policy" no console	Falta `@CrossOrigin` ou a porta está errada.
Erro de template/view não encontrada	Usou `@Controller` em vez de `@RestController`.
Todo endpoint dá 404	Application fora da raiz do pacote, ou rota errada (prefixo + mapping).
"declared package does not match"	Linha `package` diferente da pasta (ou faltando).
"cannot be resolved to a type"	Import faltando (`java.util.List`, não java.awt). Use Ctrl + .
Requisições infinitas	Faltou o `[]` no useEffect.
Filtro não atualiza	Lista filtrada guardada em estado em vez de calculada.
Warning de key no console	map sem `key` (use o id, não o índice).
Modal fecha ao clicar dentro	Faltou `stopPropagation` na caixa.
Função roda sozinha ao renderizar	`onClick={f(x)}` → trocar por `onClick={() => f(x)}`.

10 MINUTOS FINAIS

22	**Teste de ponta a ponta** Back e front rodando juntos: listar, pesquisar, filtrar, abrir detalhe, trocar de aba.
23	**Desligar o Spring** A mensagem de erro precisa aparecer no React.
24	**Console limpo (F12)** Sem CORS, sem warning de key, sem erro vermelho.
25	**Compila?** Projeto que não roda vale no máximo 50%. Melhor entregar menos funcionando do que tudo quebrado.

COMANDOS NO CMD (WINDOWS)

Conferir instalações

```
java -version
node -v
npm -v
```

Rodar o back-end (pasta com o pom.xml)

```
cd caminho\do\backend
mvnw spring-boot:run

rem com Maven instalado: mvn spring-boot:run
rem limpar e recompilar: mvnw clean package
```

Criar e rodar o front-end

```
npm create vite@latest nome-do-app -- --template react
cd nome-do-app
npm install
npm run dev
```

Projeto React já existente / baixado

```
cd caminho\do\frontend
npm install        (só na primeira vez)
npm run dev
```

Testar a API sem o React

```
curl http://localhost:8080/api/itens
curl http://localhost:8080/api/itens/1
curl "http://localhost:8080/api/itens?tipo=X&texto=ab"
```

"Port 8080 already in use"

```
netstat -ano | findstr :8080
taskkill /PID numero_do_pid /F
```

Parar um servidor: **Ctrl + C** no terminal. Back e front rodam em **dois terminais** ao mesmo tempo. No VS Code: Terminal → New Terminal (ou o botão +).

CÓDIGO DE REFERÊNCIA — BACK-END (nomes genéricos: Item / ItemService)

Application — na RAIZ do pacote

```
package com.example.projeto;

@SpringBootApplication
public class ProjetoApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProjetoApplication.class, args);
    }
}
```

Controller REST — o esqueleto completo

```
package com.example.projeto.controller;          // = pasta .../projeto/controller

import java.util.List;                            // java.util, NÃO java.awt
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController                                   // retorna JSON (não view)
@RequestMapping("/api")                           // prefixo de todas as rotas
@CrossOrigin(origins = "http://localhost:5173")   // libera o React
public class ItemController {

    private final ItemService service;            // injeção pelo construtor
    public ItemController(ItemService service) { this.service = service; }

    // GET /api/itens?tipo=X&texto=ab   (filtros opcionais -> @RequestParam)
    @GetMapping("/itens")
    public List<Item> listar(@RequestParam(required = false) String tipo,
                             @RequestParam(required = false) String texto) {
        return service.filtrar(tipo, texto);      // CONFERIR a ordem no service!
    }

    // GET /api/itens/5   (dado no caminho -> @PathVariable; pode não existir -> 404)
    @GetMapping("/itens/{id}")
    public ResponseEntity<Item> buscar(@PathVariable int id) {
        return service.buscarPorId(id)            // Optional<Item>
                .map(ResponseEntity::ok)                    // tem  -> 200
                .orElse(ResponseEntity.notFound().build()); // vazio -> 404
    }
}
```

Se a prova pedir Thymeleaf (como na AV1) — a diferença

```
@Controller                                          // retorna o NOME da view
public class PaginaController {
    @GetMapping("/")
    public String index(@RequestParam(required = false) String texto, Model model) {
        model.addAttribute("itens", service.filtrar(null, texto));
        model.addAttribute("textoSelecionado", texto == null ? "" : texto);
        return "index";                              // -> templates/index.html
    }
}
```

CÓDIGO DE REFERÊNCIA — FRONT-END

api.js — fetch com tratamento de erro

```
const BASE_URL = "http://localhost:8080/api";

async function getJson(caminho) {
  const resposta = await fetch(`${BASE_URL}${caminho}`);
  if (!resposta.ok) throw new Error(`Erro ${resposta.status}`);  // fetch não falha em 404
  return resposta.json();                                         // JSON -> objeto JS
}

export const buscarItens = () => getJson("/itens");
export const buscarItem  = (id) => getJson(`/itens/${id}`);
```

App.jsx — estados, carga inicial e estado derivado

```
import { useEffect, useState } from "react";
import { buscarItens } from "./api";

export default function App() {
  const [itens, setItens] = useState([]);            // começa [] (não null)
  const [carregando, setCarregando] = useState(true);
  const [erro, setErro] = useState(null);
  const [busca, setBusca] = useState("");
  const [tipo, setTipo] = useState("");              // "" = todos
  const [idSelecionado, setIdSelecionado] = useState(null);

  useEffect(() => {                                   // roda UMA vez por causa do []
    buscarItens()
      .then(setItens)
      .catch((e) => setErro(e.message))
      .finally(() => setCarregando(false));
  }, []);

  // ESTADO DERIVADO: calculado a cada render, sem useState
  const termo = busca.trim().toLowerCase();
  const filtrados = itens.filter((i) =>
    (termo === "" || i.nome.toLowerCase().includes(termo) || String(i.id) === termo) &&
    (tipo === "" || i.tipos.includes(tipo))
  );

  if (carregando) return <p>Carregando...</p>;
  if (erro) return <p>Erro: {erro}</p>;

  return (
    <>
      <SearchBar busca={busca} onBuscaChange={setBusca} />
      <p>{filtrados.length} item(ns) encontrado(s)</p>
      {filtrados.length === 0
        ? <p>Nenhum encontrado.</p>
        : <ItemList itens={filtrados} onSelecionar={setIdSelecionado} />}
      {idSelecionado && <Detalhes id={idSelecionado} onFechar={() => setIdSelecionado(null)} />}
    </>
  );
}
```

Input e select controlados

```
function SearchBar({ busca, onBuscaChange }) {
  return (
    <input value={busca}
      onChange={(e) => onBuscaChange(e.target.value)} />
  );
}

function Filtro({ opcoes, valor, onMudar }) {
  return (
    <select value={valor}
      onChange={(e) => onMudar(e.target.value)}>
      <option value="">Todos</option>
      {opcoes.map((o) => (
        <option key={o} value={o}>{o}</option>
      ))}
    </select>
  );
}
```

Lista com map + key, card recebendo props

```
function ItemList({ itens, onSelecionar }) {
  return (
    <div className="grade">
      {itens.map((i) => (
        <ItemCard key={i.id} item={i}
          onClick={() => onSelecionar(i.id)} />
      ))}
    </div>
  );
}

function ItemCard({ item, onClick }) {
  return (
    <article className="card" onClick={onClick}>
      <h2>{item.nome}</h2>
    </article>
  );
}
```

Modal: fetch por id + fechar fora/dentro

```
function Detalhes({ id, onFechar }) {
  const [item, setItem] = useState(null);

  useEffect(() => {                          // roda de novo sempre que o id mudar
    setItem(null);
    buscarItem(id).then(setItem);
  }, [id]);

  return (
    <div className="overlay" onClick={onFechar}>                    {/* clicar fora fecha */}
      <div className="modal" onClick={(e) => e.stopPropagation()}>  {/* clicar dentro não */}
        <button onClick={onFechar}>x</button>
        {!item ? <p>Carregando...</p> : <h2>{item.nome}</h2>}
      </div>
    </div>
  );
}
```

Truques úteis

```
// ordenar sem mutar o estado
const ordenados = [...lista]
  .sort((a, b) => b.nivel - a.nivel);

// mapa id -> objeto (IDs vindos da API)
const porId = Object.fromEntries(
  itens.map((i) => [i.id, i]));
porId[3].nome

// tipos únicos sem API (modo só React)
[...new Set(itens.flatMap((i) => i.tipos))]
  .sort()

// classe sem acento: "Água" -> "agua"
s.normalize("NFD")
  .replace(/[\u0300-\u036f]/g, "")
  .toLowerCase()

// importar JSON local (modo só React)
import dados from "./data/arquivo.json";
```

CSS essencial

```
.grade {
  display: grid;
  grid-template-columns:
    repeat(auto-fill, minmax(220px, 1fr));
  gap: 16px;
}
.card:hover { transform: translateY(-4px); }

.overlay {
  position: fixed; inset: 0;
  background: rgba(0, 0, 0, 0.7);
  display: flex;
  align-items: center; justify-content: center;
}

.trilho { height: 10px; overflow: hidden; }
/* barra: style={{ width: `${v / 150 * 100}%` }} */
```